IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

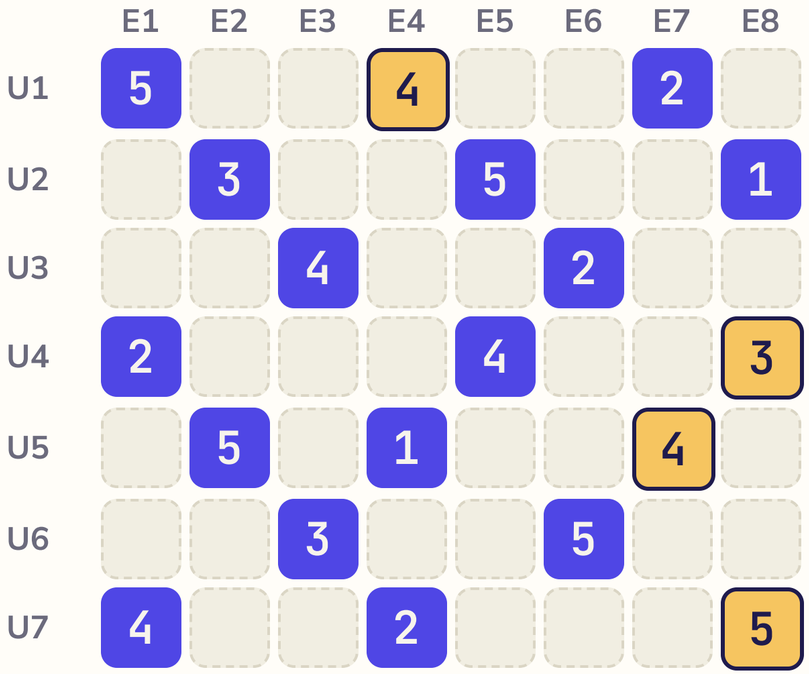
Recomendar experiencias a cada huésped

Popularidad, contenido, filtrado colaborativo y factorización; RMSE y precision@3

Sprint 9 · S09_03 · cuaderno: 50 min en clase (ejercicios 1 a 3)

LA IDEA · SECCIÓN 1

Una tabla casi vacía



■ nota que se usa · ■ nota escondida (prueba) · □ casilla vacía

Cada fila es un **usuario**, cada columna una **experiencia** y cada casilla, la nota de 1 a 5. Casi todas están **vacías**: nadie ha probado todo.

24 %

de las casillas tienen nota: **1786 de 7500** en el cuaderno.

Se esconde el 20 % de las notas (446): el recomendador nunca las ve al aprender.

CÓMO SE EXAMINA · SECCIÓN 1

Esconder notas y medir cuánto se equivoca

```
prueba = valoraciones.sample(frac=0.2, random_state=SEMILLA)    # 446 notas escondidas
entrenamiento = valoraciones.drop(prueba.index)                 # 1786 notas con las que aprende
R = entrenamiento.pivot(index="id_usuario", columns="id_experiencia", values="valoracion")
```

nota escondida	real	predicción	error	error²
A	5	4	−1	1
B	4	4	0	0
C	2	4	+2	4
D	3	2	−1	1

1,22

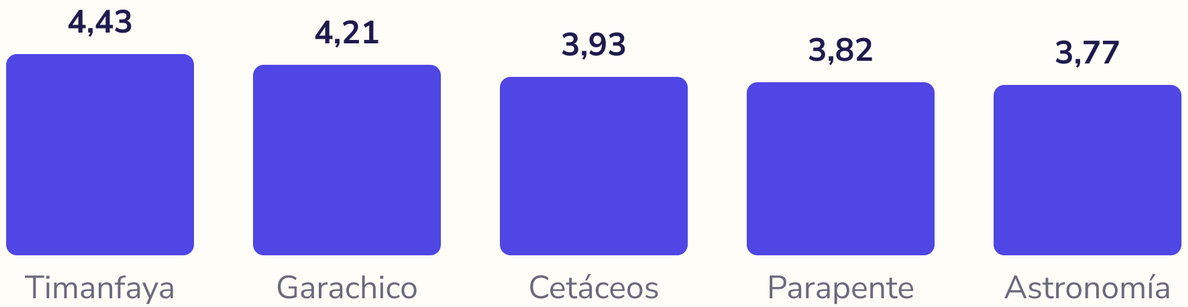
RMSE del ejemplo: **√(6 ÷ 4)** estrellas

Punto de partida

Predecir siempre la **media global (2,97)** da un RMSE de **1,275**.

MÉTODO 1 · POPULARIDAD · EJERCICIO 1 · CUADERNO (CLASE)

Lo mejor valorado, para todos por igual



RMSE 1,029

Mejora la media global (1,275) sin personalizar nada.

Arranque en frío

Es la única opción con un **huésped nuevo**.

```
media_item = R.mean()                          # nota media de cada experiencia (solo entrenamiento)
P_pop = np.tile(media_item.values, (300, 1))   # la misma fila para los 300 usuarios
rmse_pop = evaluar_rmse(P_pop)                 # 1.029
```

MEDIR LA LISTA · EJERCICIO 2 · CUADERNO (CLASE)

precision@3: ¿acierta con la lista corta?

El cliente no ve notas: ve **3 sugerencias**. Se recomienda solo entre lo **no valorado** y se acierta si en la prueba le gustó (**4 estrellas o más**).

Usuario inventado	recomendada	¿le gustó?
Le gustaron: 3, 8 y 14	8	sí
	11	no
	20	no

precision@3 = 1 ÷ 3

De 3 recomendadas, le gustó **1**: **0,33**.

popularidad **0,125** · azar **0,081** · **115** usuarios evaluables

```
puntuacion = P[u - 1].astype(float).copy()
puntuacion[M[u - 1]] = -np.inf                    # oculta lo que ya valoró
recomendadas = np.argsort(-puntuacion)[:k] + 1    # las k de mayor puntuación
```

MÉTODO 2 · CONTENIDO · EJERCICIO 3 · CUADERNO (CLASE)

Si te gusta el mar, más experiencias de mar

El perfil de cada usuario es cuánto puntúa **por encima o por debajo de lo normal** cada categoría. Residuo = su nota − la nota media de esa experiencia.

Ejemplo inventado	valoraciones	suma de residuos	perfil = suma ÷ (n + 3)
mar	3	+3,5	0,58
aventura	1	+2,0	0,50
cultura	0	0	0,00

El «+3» es prudencia

Con **una sola** valoración no nos fiamos: sale 0,50 y no 2,0.

RMSE 1,066 · precision@3 0,130

Peor que la popularidad en RMSE (1,029).

```
perfil_usuario = (C @ F) / (M.astype(float) @ F + 3)     # 300 usuarios × 7 categorías
P_cont = media_item_v[None, :] + perfil_usuario @ F.T    # media + afinidad
```

MÉTODO 3 · COLABORATIVO · EJERCICIO 4 · CUADERNO (CASA)

La gente como tú también valoró…

Sin describir las experiencias: dos experiencias son **parecidas** si la misma gente las puntúa por encima o por debajo de lo normal. La similitud es el **coseno** entre columnas de residuos.

usuario	exp. A	exp. B	exp. C
U1	+1	+1	−1
U2	−1	−1	+1
U3	+1	0	0
U4	0	+1	−1

A y B: **0,67** (van a la par) · B y C: **−1** (opuestas) · A y C: −0,67



Las 3 más parecidas al **Ascenso al Teide**. Anaga, también de naturaleza, sale con **−0,085**.

item-item: RMSE 0,895 y precision@3 0,171 · user-user: RMSE 0,865 y precision@3 0,148

MÉTODO 4 · FACTORIZACIÓN · EJERCICIO 5 · CUADERNO (CASA)

Gustos ocultos que se aprenden de los datos

Cada usuario tiene unos pocos **gustos ocultos** y cada experiencia una **intensidad** en cada uno. El algoritmo los aprende con las casillas conocidas y **rellena las vacías**.

$$\begin{pmatrix} 1{,}2 & -0{,}4 \end{pmatrix} \times \begin{pmatrix} 0{,}9 \\ 0{,}5 \end{pmatrix} = \begin{pmatrix} 0{,}88 \end{pmatrix}$$

gustos del usuario

intensidades de la experiencia

gustos · intensidades

Nota prevista (ejemplo)	
media global	2,97
+ sesgo del usuario (generoso)	+0,30
+ sesgo de la experiencia	+0,40
+ gustos · intensidades	+0,88
= nota prevista	4,55

RMSE 0,860 · precision@3 0,180

El mejor de los cuatro métodos en las dos medidas.

gustos ocultos k	2	3	4	6
RMSE	0,875	0,860	0,867	0,870

COMPARAR MÉTODOS · SECCIÓN 6 (CASA)

Cambiar de métrica cambia el orden

Método	Qué mira	RMSE (menos es mejor)	precision@3 (más es mejor)
Media global	nada	1,275	—
Popularidad	lo mejor de todos	1,029	0,125
Contenido	la categoría	1,066	0,130
Item-item	experiencias parecidas	0,895	0,171
User-user	usuarios parecidos	0,865	0,148
Factorización (k = 3)	gustos ocultos	0,860	0,180

El orden cambia

Contenido pierde con popularidad en RMSE pero la supera en precision@3. User-user es 2.º en RMSE y 3.º en precision@3.

115 usuarios evaluables

Una diferencia de **0,01** en precision@3 es **ruido**: no se la vendas al cliente.

MINI-RETO · REFLEXIÓN 3 (CASA)

recomendar(id_usuario): la lista para un cliente

```
def recomendar(id_usuario, n=3, P=P_mf):
    if not 1 <= id_usuario <= len(M):
        raise ValueError(...)
    puntuacion = P[id_usuario - 1].astype(float).copy()
    puntuacion[M[id_usuario - 1]] = -np.inf
    ids = np.argsort(-puntuacion)[:n] + 1
```

usuario	experiencia	nota prevista
1	Parque Timanfaya	4,69
1	Ruta por Anaga	4,34
1	Zoo y jardines	3,50
2	Observación de cetáceos	4,39
2	Astronomía en el Roque	4,19
2	Visita a Garachico	3,96

Para el reto

Allí se compara con la popularidad el método **item-item**: **recomendar(..., P=P_item)**.

Sesgo de popularidad

Con popularidad, las listas de los 300 usuarios usan solo **8 de las 25** experiencias; con item-item, **18**.

HOY HAS APRENDIDO

Esconder notas para saber si acierta

Cuatro métodos

Popularidad (la referencia), contenido, colaborativo y factorización

Dos medidas

RMSE mide la nota; **precision@k** mide la lista que ve el cliente

Con cuidado

Pocos usuarios evaluables: una diferencia pequeña es ruido; y cuidado con el sesgo de popularidad

Siguiente: S09_04 · aprendizaje por refuerzo (Q-learning)